

FCM II Graded Homework 3

Charles Ohanian

March 21, 2025

1 Executive Summary

This report evaluates various interpolation techniques, including global and piecewise polynomial interpolation, as well as interpolatory cubic splines, to approximate given functions and data sets. Piecewise interpolating polynomials of various degrees, cubic Hermite interpolating polynomials, and interpolatory cubic splines were used to analyze interpolation accuracy for different functions, including a sinusoidal function. The study also applied an interpolatory cubic spline to a discrete dataset to estimate function values and their derivatives. Results demonstrate the effectiveness of these interpolation techniques, with errors primarily influenced by mesh selection, function behavior, and machine error.

2 Statement of the Problem

The goal of this study is to analyze and compare different interpolation methods for function approximation and data fitting. Specifically, the report investigates global and piecewise polynomial interpolation, cubic Hermite interpolation, and interpolatory cubic splines, to approximate given functions and discrete datasets. The accuracy and error behavior of these methods are examined under various conditions, such as uniform and Chebyshev meshes, to assess their effectiveness in representing smooth and oscillatory functions. The study also extends to estimating derivatives and transformed function values from interpolated data.

3 Description of the Mathematics

This section outlines the mathematical concepts which will be of use for this assignment. Piecewise polynomial interpolation and interpolatory cubic splines will be discussed.

3.1 Piecewise Polynomial Interpolation

Piecewise polynomial interpolation approximates a function $f(x)$ over a global interval $[a, b]$ by constructing separate interpolants over a set of sub-intervals. Given n sub-intervals, interpolation nodes are selected within each sub-interval, either uniformly or according to Chebyshev spacing, to minimize interpolation error. When constructing non-Hermite piecewise interpolating polynomials, the coefficients can be computed using divided differences, as discussed in the previous assignment. The polynomials can be evaluated using Horner's

rule, also as previously discussed. When constructing a cubic Hermite interpolating polynomial, the four necessary coefficients can be computed explicitly. The first coefficient is f_i , the second coefficient is $f[x_i, x_i] = f'_i$, and the third and fourth coefficients are as follows:

$$f[x_i, x_i, x_{i+1}] = \frac{f_{i+1} - f_i}{(x_{i+1} - x_i)^2} - \frac{f'_i}{(x_{i+1} - x_i)}$$

$$f[x_i, x_i, x_{i+1}, x_{i+1}] = \frac{f'_{i+1} + f'_i}{(x_{i+1} - x_i)^2} - \frac{2(f_{i+1} - f_i)}{(x_{i+1} - x_i)^3}.$$

The polynomials can then be evaluated using the following:

$$H_{3,i}(x) = f_i + f'_i(x - x_i) + f[x_i, x_i, x_{i+1}](x - x_i)^2 + f[x_i, x_i, x_{i+1}, x_{i+1}](x - x_i)^2(x - x_{i+1}).$$

3.2 Interpolatory Cubic Splines

A cubic spline is a piecewise polynomial of degree 3 that ensures smooth transitions between adjacent sub-intervals. This is enforced by the requirement that $s(x)$ be twice continuously differentiable over $[a, b]$, i.e., $s'(x)$ and $s''(x)$ exist and are continuous. As described in Dr. Gallivan's class notes, when utilizing the s''_i parameterization, a tridiagonal system can be solved to obtain the s''_i 's. Then, these can be used in the following formula to evaluate the spline:

$$p_i(x) = s''_{i-1} \frac{(x_i - x)^3}{6h_i} + s''_i \frac{(x - x_{i-1})^3}{6h_i} + \gamma_{i-1}(x - x_{i-1}) + \tilde{\gamma}_{i-1},$$

where γ_{i-1} and $\tilde{\gamma}_{i-1}$ can be computed explicitly as

$$\gamma_{i-1} = \frac{f_i - f_{i-1}}{h_i} - \frac{h_i}{6}(s''_i - s''_{i-1})$$

$$\tilde{\gamma}_{i-1} = f_{i-1} - s''_{i-1} \frac{h_i^2}{6}.$$

When utilizing the B-spline basis parameterization, the function $f(x)$ can be approximated as

$$s(x) = \sum_{i=0}^n \alpha_i B_i(x),$$

where the α 's are obtained by solving a near-tridiagonal system and the B_i 's are computed based on a center reference point x_i and the support $[x_{i-2}, x_{i+2}]$, as described in Dr. Gallivan's class notes.

4 Description of the Algorithm and Implementation

All routines and experiments were developed and run in Google Colab, a cloud-based platform provided by Google that allows users to write and execute Python code in a Jupyter Notebook environment.

Routines were developed for a single global interpolating polynomial using Barycentric form 1, a piecewise interpolating polynomial, and two versions of an interpolatory cubic spline. The global interpolation algorithm will not be discussed here, as it remains unchanged from the previous assignment.

4.1 Piecewise Interpolating Polynomial

The `piecewise_interpolating_polynomial` function constructs a piecewise polynomial $g_d(x)$ to approximate a given function $f(x)$ over $[a, b]$ using the Newton basis construction. The routine includes the option of constructing a piecewise cubic Hermite interpolating polynomial for further smoothness. The routine first divides the global interval into a specified number (`n_intervals`) of uniform sub-intervals, storing endpoints in nodes. Each sub-interval is assigned interpolation points based on the chosen method: uniform or Chebyshev (created using the `create_mesh` function described in the previous assignment). If Hermite interpolation is selected, both function values and derivatives are computed at the nodes. The separate `hermite_divided_differences` function then computes the four coefficients needed for a cubic Hermite interpolating polynomial. If Hermite interpolation is not desired, the `divided_differences` function (as described in the previous assignment) generates the polynomial coefficients for Newton interpolation.

The function `gd(x)` evaluates $g_d(x)$. First, the routine checks whether the given x point is numerically close to any of the mesh points. If this is the case, the code directly returns $f(x)$ with no further computation. If x is not sufficiently close to a node, the routine determines the sub-interval containing x and uses the previously stored coefficients to compute the interpolated value. If Hermite interpolation is used, the `evaluate_newton_polynomial_hermite` function is applied, which explicitly evaluates a cubic polynomial using precomputed coefficients. Otherwise, `evaluate_newton_polynomial` computes Newton's polynomial efficiently via nested multiplication using Horner's rule.

Since the algorithm is restricted to polynomials of degree at most three, the computational complexity is significantly reduced. Constructing the divided differences for Newton interpolation runs in constant time $O(1)$ per sub-interval because, at most, four coefficients need to be computed. Hermite interpolation similarly operates in $O(1)$ time per sub-interval, as it always computes exactly four coefficients using predefined formulas. Evaluation is also efficient, as both Newton and Hermite polynomial evaluations require at most three iterations, making them $O(1)$ operations. Since each sub-interval is handled separately, constructing the full piecewise interpolant scales linearly with the number of sub-intervals, giving an overall complexity of $O(m)$, where m is the number of sub-intervals. Evaluating the function at a given point is always $O(1)$. The space complexity is $O(n)$, as only interpolation nodes and coefficients are stored.

4.2 Interpolatory Cubic Spline Code 1

The given algorithm constructs and evaluates an interpolatory cubic spline using the s_i'' parameterization. The spline is determined by solving a tridiagonal system to compute the second derivatives at the interpolation points and then evaluating the piecewise cubic function in each sub-interval. The function `tridiagonal_system` efficiently solves the linear system required for cubic spline interpolation. It takes in as parameters the constant value that is used for the main diagonal, the vector of the subdiagonal elements, the vector of the superdiagonal elements, and the right-hand side vector. It performs a forward sweep to compute intermediate coefficients α and β , followed by a back-substitution step to find the solution

vector x . Since the matrix is tridiagonal, this approach runs in $O(n)$ time, significantly reducing computational cost compared to Gaussian elimination's $O(n^3)$ complexity.

The `evaluate_spline` function computes the interpolated values by iterating through the provided evaluation points x and determining which sub-interval $[a_i, b_i]$ each point belongs to. The spline is evaluated using the standard cubic spline formula, where the second derivatives s_i'' contribute to the cubic terms, while the precomputed coefficients γ_i and $\tilde{\gamma}_i$ account for linear adjustments. This function ensures efficient evaluation in $O(1)$ time per query by leveraging precomputed values and avoiding redundant computations. Since only $O(n)$ storage is required for the tridiagonal system and spline coefficients, the overall space complexity remains $O(n)$.

4.3 Interpolatory Cubic Spline Code 2 (B-Spline basis)

This algorithm constructs and evaluates an interpolatory cubic spline using the B-spline basis parameterization. Unlike the previous cubic splines routine that utilized second derivative continuity, this approach represents the interpolant as a linear combination of B-spline basis functions, with coefficients α determined by solving a linear system. The function `construct_near_tridiagonal_matrix` builds a nearly tridiagonal system that encodes the smoothness constraints of the B-spline basis. The resulting matrix has a mainly tridiagonal structure with additional elements in the first and last rows, enforcing the boundary conditions. This matrix is efficiently structured and an efficient solver requiring only $O(n)$ computations and $O(n)$ space could likely be implemented. However, this method was unfortunately not discovered for this assignment, so `np.linalg.solve` was used to solve the system.

Once the coefficients α are determined, `evaluate_B_spline` performs interpolation by summing the weighted cubic B-spline basis functions over the relevant sub-interval. It determines the correct interval for each test point and evaluates the B-spline using piecewise cubic polynomials, maintaining $O(1)$ evaluation complexity per query. In terms of space complexity, `evaluate_B_spline` primarily stores the `interp` list, which holds the interpolated values for each query point, requiring $O(n)$ space. Additionally, it accesses the precomputed coefficient vector α , also of size $O(n)$, meaning the overall space complexity is $O(n)$.

5 Description of the Experimental Design and Results

5.1 Task 1

In this section, we will empirically validate the correct functioning of the algorithms. To accomplish this, we will run tests on two different functions: a cubic polynomial and a sine function. The cubic polynomial is included to demonstrate the expected accuracy of the routines with a lower-degree function, while the sinusoid is included to test the routines with something a bit more oscillatory and complicated. For each function, figures will display the results of interpolating/approximating it with the following strategies:

1. Barycentric 1 with a mesh of Chebyshev points of the second kind
2. Piecewise linear with a uniform and Chebyshev 2 mesh

3. Piecewise quadratic with with a uniform and Chebyshev 2 mesh
4. Piecewise cubic with a uniform and Chebyshev 2 mesh
5. Cubic Hermite with a uniform and Chebyshev 2 mesh
6. Interpolatory cubic spline (s''_i parameterization) with a uniform and Chebyshev 2 mesh
7. Interpolatory cubic spline (B-spline basis parameterization) with a uniform mesh

5.1.1 Interpolation Results for a Cubic Polynomial

In this section, approximation and interpolation accuracy is discussed for $f(x) = 4x^3 + 3x^2 + 2x + 1$. Figure 1 shows the results of interpolating $f(x)$ with a single global 5th degree polynomial. Barycentric form 1 is used to obtain the interpolant. The interpolation error is on the order of 10^{-14} , which suggests that the interpolant is highly accurate. Since the function being interpolated is a cubic polynomial and the interpolating polynomial is degree 5, the interpolation should be exact in exact arithmetic. The observed error is due to floating-point precision limits.

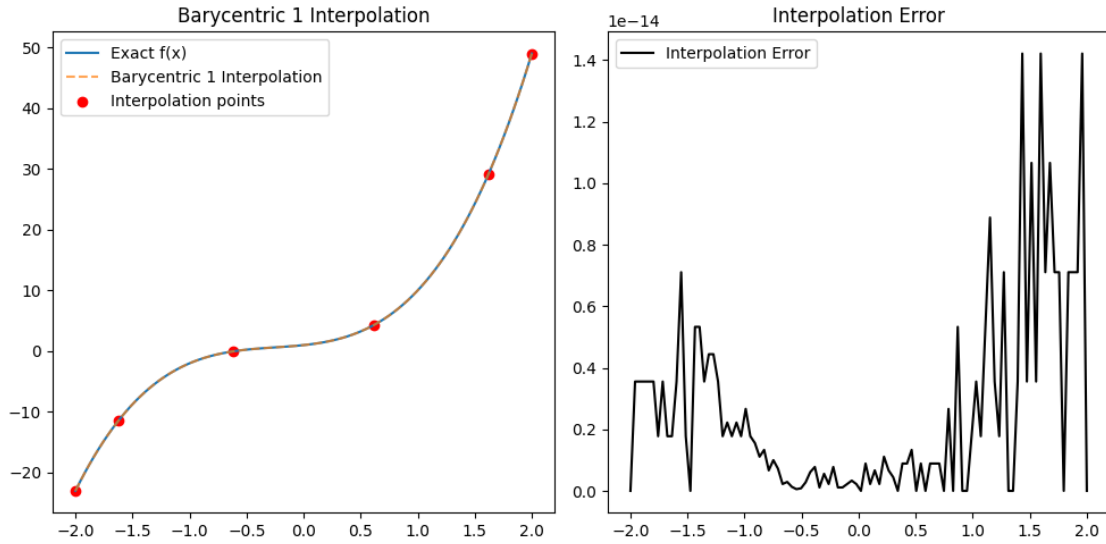


Figure 1. Global interpolating polynomial $p_5(x)$ vs $f(x)$ and the associated interpolation error. Barycentric 1 is used with a Chebyshev mesh broken into 5 intervals (6 mesh points).

Figure 2 shows the results of interpolating $f(x)$ with a piecewise interpolating polynomial of various degrees, using a uniform mesh on each sub-interval. It also shows the results when using a cubic Hermite interpolating polynomial. It can clearly be observed that the error decreases as the degree of the interpolant increases, confirming the expected theoretical behavior: higher-degree interpolants generally yield better approximations. The dramatic drop that occurs when going from quadratic to cubic makes sense because $f(x)$ is cubic, so a piecewise cubic interpolating polynomial should approximate it accurately. Also of important note are the periodic downward spikes in error that can be seen for the lower-degree interpolants. The approximation is more accurate near the mesh points, while in

between them, the interpolant deviates from $f(x)$ more due to the limited flexibility of the lower-degree approximations.

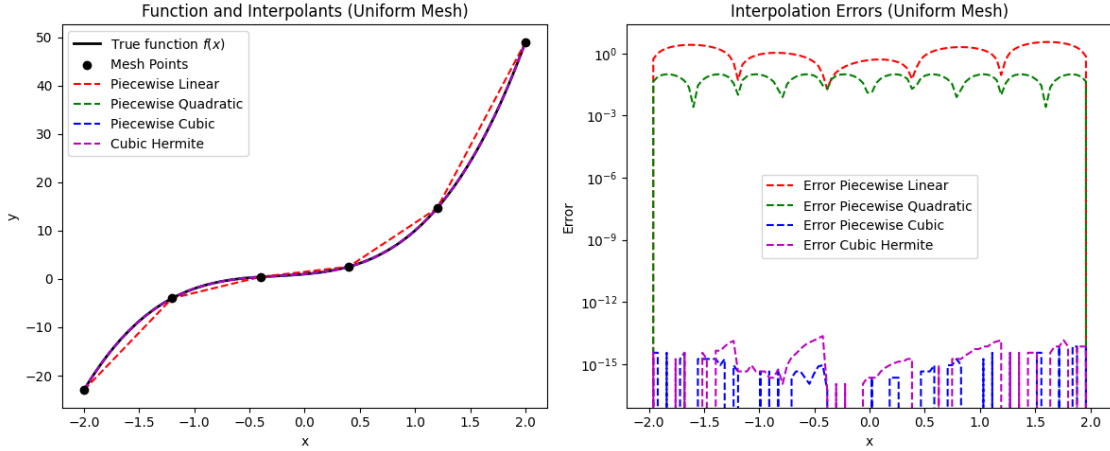


Figure 2. Piecewise interpolating polynomial $g_d(x)$ vs $f(x)$ and the associated interpolation error. Linear, quadratic, and cubic piecewise interpolating polynomials, as well as a cubic Hermite interpolating polynomial are tested using a uniform mesh broken into 5 intervals (6 mesh points).

Figure 3 shows the results of interpolating $f(x)$ with a piecewise interpolating polynomial of various degrees, using a Chebyshev mesh on each sub-interval. It also shows the results when using a cubic Hermite interpolating polynomial. We see very similar results when using either the uniform or Chebyshev mesh on the sub-intervals. All of the behavior commented on above remains the same. It is likely that this difference becomes more noticeable as the degree increases, however for this assignment, the degree was limited to 3.

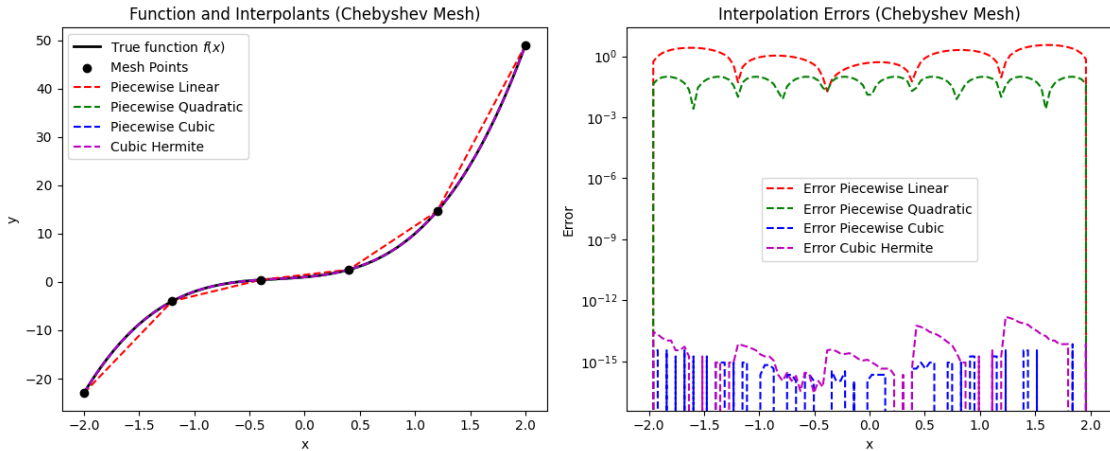


Figure 3. Piecewise interpolating polynomial $g_d(x)$ vs $f(x)$ and the associated interpolation error. Linear, quadratic, and cubic piecewise interpolating polynomials, as well as a cubic Hermite interpolating polynomial are tested using a Chebyshev mesh broken into 5 intervals (6 mesh points).

Figure 4 compares the exact function $f(x)$ with the interpolatory cubic spline $s(x)$ when using a uniform mesh. The error is nearly at machine precision across the domain, confirming the expected accuracy of the cubic spline interpolant. Since the function $f(x)$ is a cubic polynomial, the cubic spline should be exact at the nodes and nearly exact in between due to numerical precision limitations. The error plot shows small oscillations due to floating-point arithmetic but remains within 10^{-15} , indicating correct computation of spline coefficients.

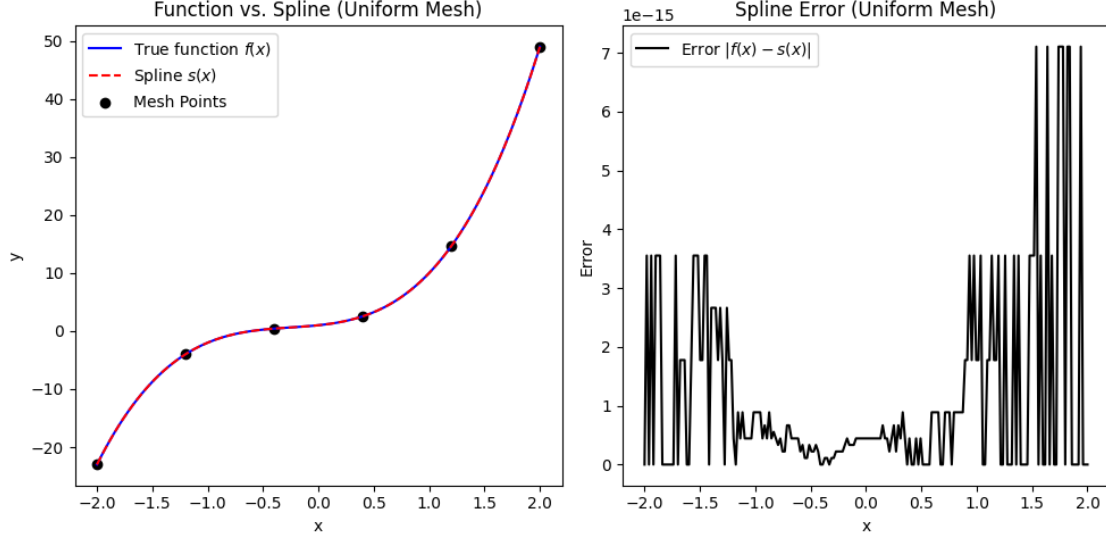


Figure 4. Interpolatory cubic spline $s(x)$ vs $f(x)$ and the associated error. The s_i'' parameterization and a uniform mesh broken into 5 intervals (6 mesh points) are used.

Figure 5 compares the exact function $f(x)$ with the interpolatory cubic spline $s(x)$ when using a Chebyshev mesh. Very similar results are seen when compared to the uniform mesh case. The error is $\sim 10^{-15}$, confirming the expected accuracy and indicating correct implementation.

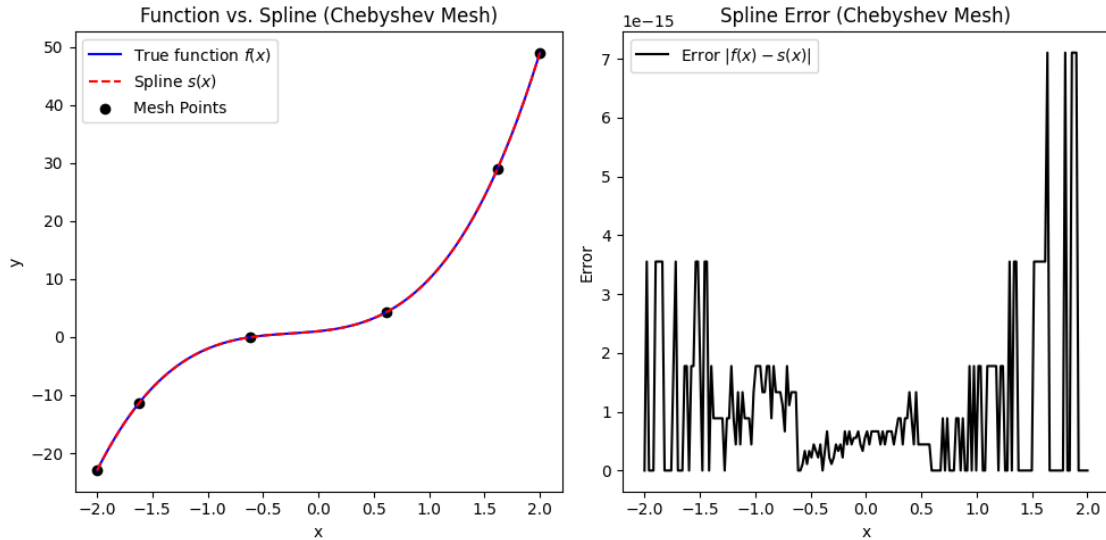


Figure 5. Interpolatory cubic spline $s(x)$ vs $f(x)$ and the associated error. The s_i'' parameterization and a Chebyshev mesh broken into 5 intervals (6 mesh points) are used.

Figure 6 compares $f(x)$ to the interpolatory cubic spline $s(x)$, where $s(x)$ is constructed using the B-spline basis parameterization. A uniform mesh is used for this approximation. The error is again very low, of order 10^{-14} , indicating that the approximation is accurate, with the only source of error being machine precision.

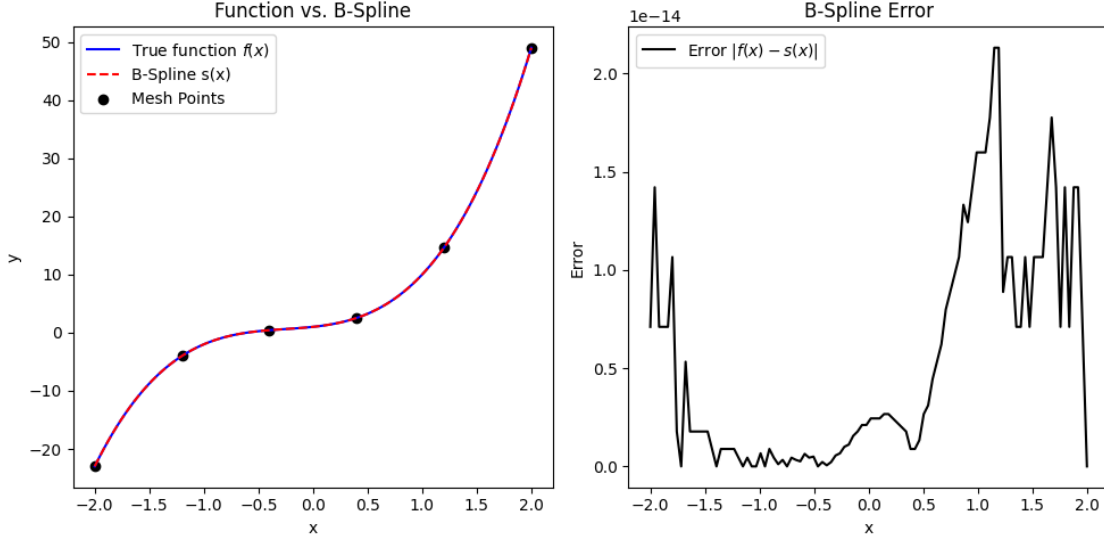


Figure 6. Interpolatory cubic spline $s(x)$ vs $f(x)$ and the associated error. The B-spline basis parameterization and a uniform mesh broken into 5 intervals (6 mesh points) are used.

5.1.2 Interpolation Results for a Sine Function

In this section, approximation and interpolation accuracy is discussed for $f(x) = \sin(x)$. Figure 7 shows the results of interpolating $f(x)$ with a single global 10th degree polynomial. Barycentric form 1 is used to obtain the interpolant. The interpolation error is on the order of 10^{-3} , meaning the approximation is relatively accurate, but not quite accurate in exact arithmetic. This highlights the difficulties of approximating a sinusoidal function with a polynomial, especially when using a single global interpolant. The oscillation seen in the error plot reflects the convergence of the interpolant to $f(x)$ near mesh points and the divergence in between mesh points.

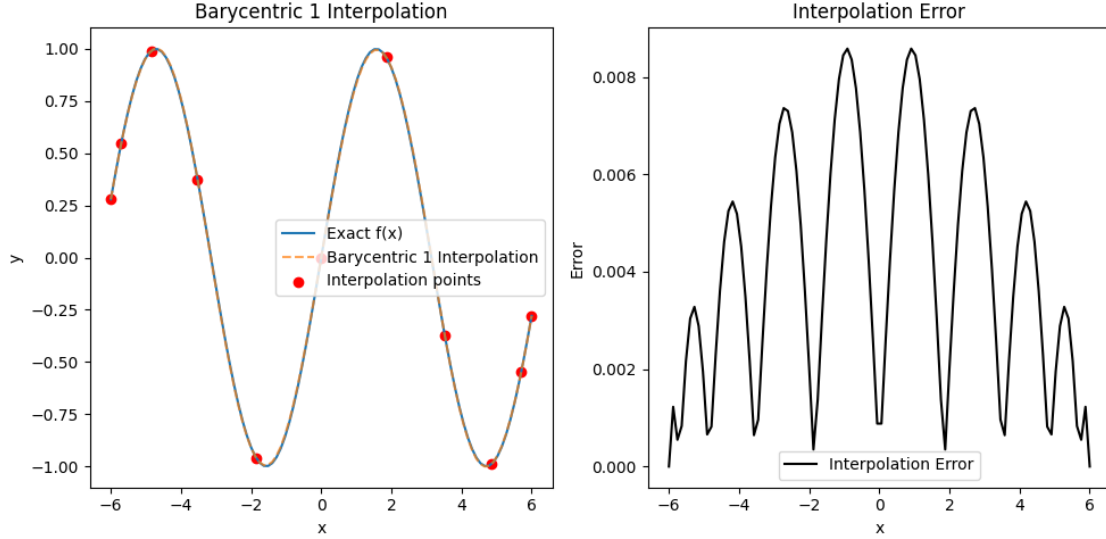


Figure 7. Global interpolating polynomial $p_{10}(x)$ vs $f(x)$ and the associated interpolation error. Barycentric 1 is used with a Chebyshev mesh broken into 10 intervals (11 mesh points).

Figure 8 shows the results of interpolating $f(x)$ with a piecewise interpolating polynomial of various degrees, using a uniform mesh on each sub-interval. It also shows the results when using a cubic Hermite interpolating polynomial. It can be observed that the error decreases as the degree of the interpolant increases, which confirms the theoretical expectation that a higher-degree interpolant should better approximate $f(x)$. Due to the periodic nature of the sine function, we can clearly see the limitations of the piecewise linear interpolant - it can not keep up with the oscillations of the function (unless using a very fine mesh). The piecewise cubic achieves slightly better accuracy than the global interpolant ($\sim 10^{-4}$). The error of the cubic Hermite interpolant seems to be a bit more oscillatory, getting very low near mesh points, but all the way up to 10^{-2} in between.

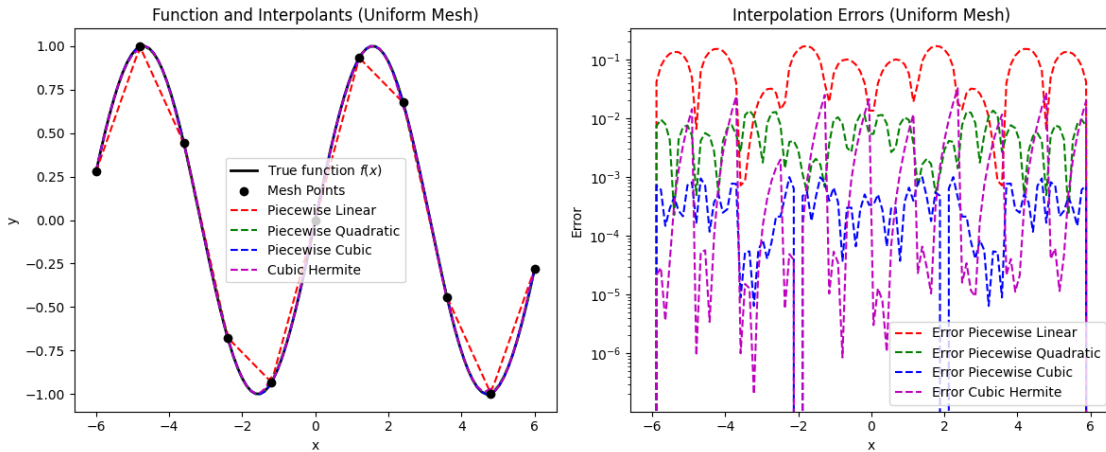


Figure 8. Piecewise interpolating polynomial $g_d(x)$ vs $f(x)$ and the associated interpolation error. Linear, quadratic, and cubic piecewise interpolating polynomials, as well as a cubic Hermite interpolating polynomial are tested using a uniform mesh broken into 10 intervals (11 mesh points).

Figure 9 shows the results of interpolating $f(x)$ with a piecewise interpolating polynomial of various degrees, using a Chebyshev mesh on each sub-interval. It also shows the results when using a cubic Hermite interpolating polynomial. Very similar results can be seen with the uniform and Chebyshev meshes. This is likely due to the low degree of the interpolant on each interval. It is expected that if the degree was increased, there would be more room to see differences between the uniform and Chebyshev meshes.

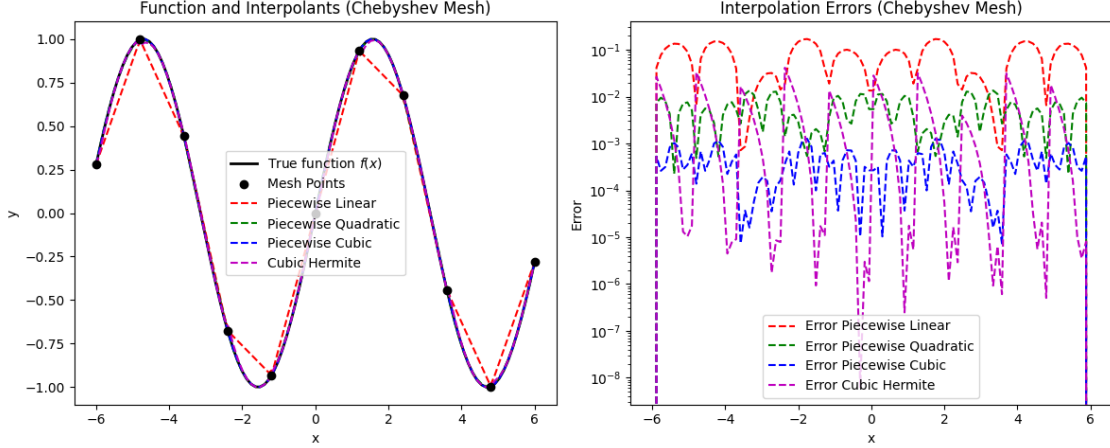


Figure 9. Piecewise interpolating polynomial $g_d(x)$ vs $f(x)$ and the associated interpolation error. Linear, quadratic, and cubic piecewise interpolating polynomials, as well as a cubic Hermite interpolating polynomial are tested using a Chebyshev mesh broken into 10 intervals (11 mesh points).

Figure 10 compares the exact function $f(x)$ with the interpolatory cubic spline $s(x)$ when using a uniform mesh. The error is similar to the case where a single global interpolant was used. This is an appropriate level of error to be seeing with an interpolatory cubic spline. As the number of mesh points increases, the spline becomes much more accurate, as expected.

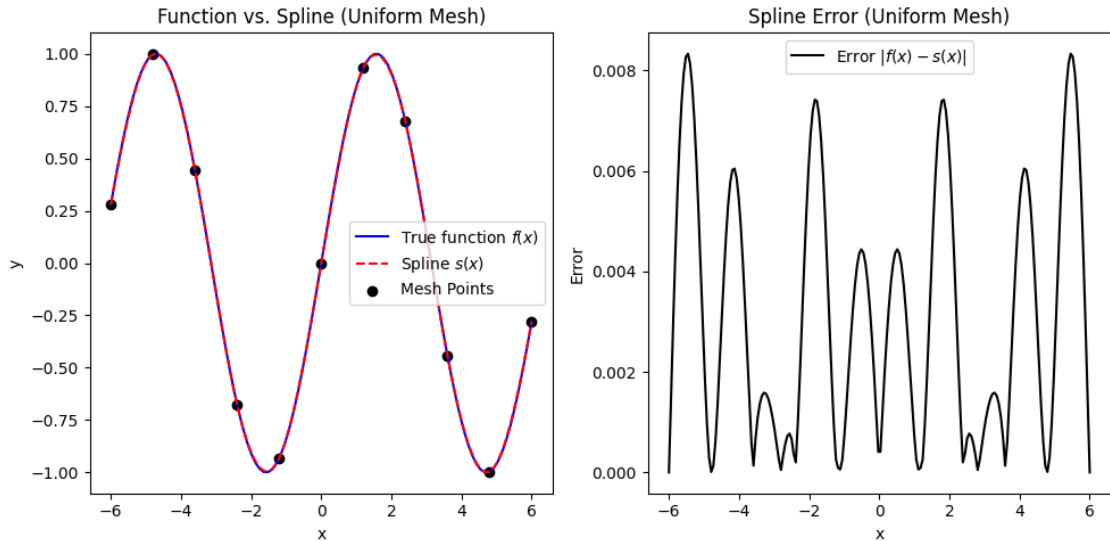


Figure 10. Interpolatory cubic spline $s(x)$ vs $f(x)$ and the associated error. The s''_i parameterization and a uniform mesh broken into 10 intervals (11 mesh points) are used.

Figure 11 compares the exact function $f(x)$ with the interpolatory cubic spline $s(x)$ when using a Chebyshev mesh. Interestingly, the error is higher when using a Chebyshev mesh compared to a uniform mesh. The error increase is concentrated in the center of the domain, which makes sense as there are less mesh points here due to the characteristic clustering at the endpoints. Again, as the number of mesh points increases, the maximum pointwise error trends down to machine epsilon.

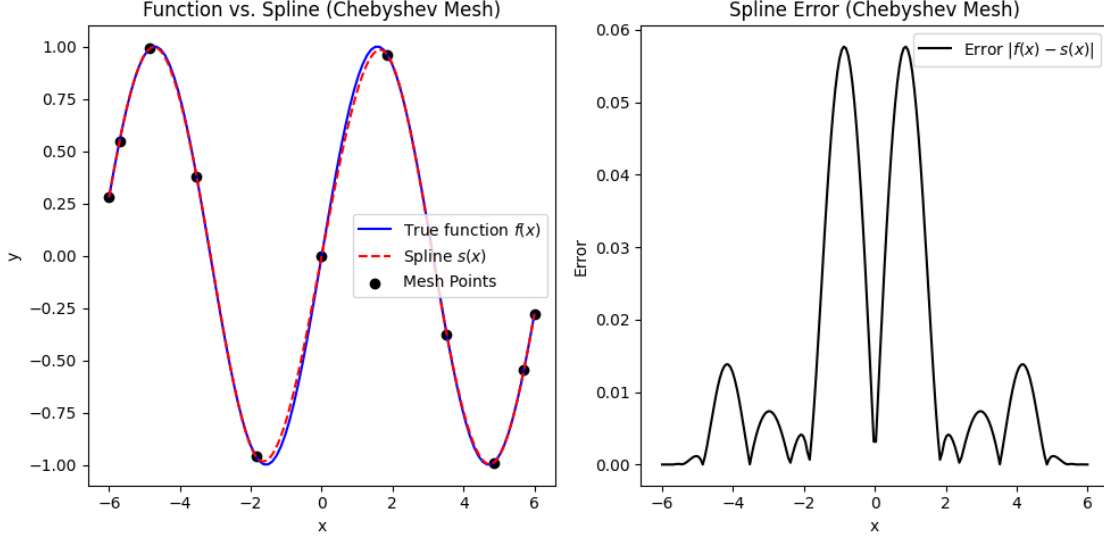


Figure 11. Interpolatory cubic spline $s(x)$ vs $f(x)$ and the associated error. The s_i'' parameterization and a Chebyshev mesh broken into 10 intervals (11 mesh points) are used.

Figure 12 compares $f(x)$ to the interpolatory cubic spline $s(x)$, where $s(x)$ is constructed using the B-spline basis parameterization. A uniform mesh is used for this approximation. The error is again of order 10^{-3} , which is an expected level for a cubic spline.

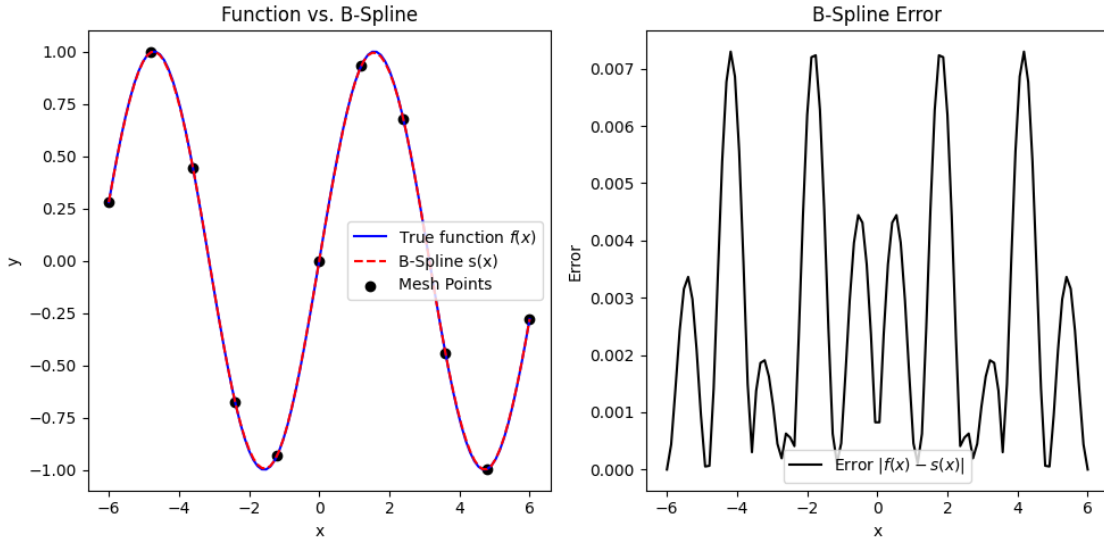


Figure 12. Interpolatory cubic spline $s(x)$ vs $f(x)$ and the associated error. The B-spline basis parameterization and a uniform mesh broken into 10 intervals (11 mesh points) are used.

5.2 Task 2

In this section, we will test the interpolatory cubic spline routine with a given dataset. We are given the following data in the form of discrete (t_i, y_i) pairs:

t_i	0.5	1.0	2.0	4.0	5.0	10.0	15.0	20.0
$y(t_i)$	0.04	0.05	0.0682	0.0801	0.0940	0.0981	0.0912	0.0857

Table 1: Table of t_i and $y(t_i)$ values

Using an interpolatory cubic spline with natural boundary conditions, that is, $s_0'' = s_n'' = 0$, $y(t)$ was estimated. Table 2 displays the computed estimates for $y(t)$ from $t_1 = 0.5$ to t_{40} with increment $\Delta t = 0.5$. Figure 13 shows this spline plotted on top of the given mesh points.

t	$y(t)$	t	$y(t)$	t	$y(t)$
0.5	0.04000	1.0	0.05000	1.5	0.06006
2.0	0.06820	2.5	0.07267	3.0	0.07476
3.5	0.07655	4.0	0.08010	4.5	0.08662
5.0	0.09400	5.5	0.09980	6.0	0.10374
6.5	0.10606	7.0	0.10702	7.5	0.10687
8.0	0.10585	8.5	0.10421	9.0	0.10221
9.5	0.10009	10.0	0.09810	10.5	0.09645
11.0	0.09513	11.5	0.09410	12.0	0.09331
12.5	0.09273	13.0	0.09230	13.5	0.09198
14.0	0.09172	14.5	0.09147	15.0	0.09120
15.5	0.09086	16.0	0.09046	16.5	0.08999
17.0	0.08948	17.5	0.08892	18.0	0.08832
18.5	0.08769	19.0	0.08704	19.5	0.08637
20.0	0.08570				

Table 2: Values of t and $y(t)$

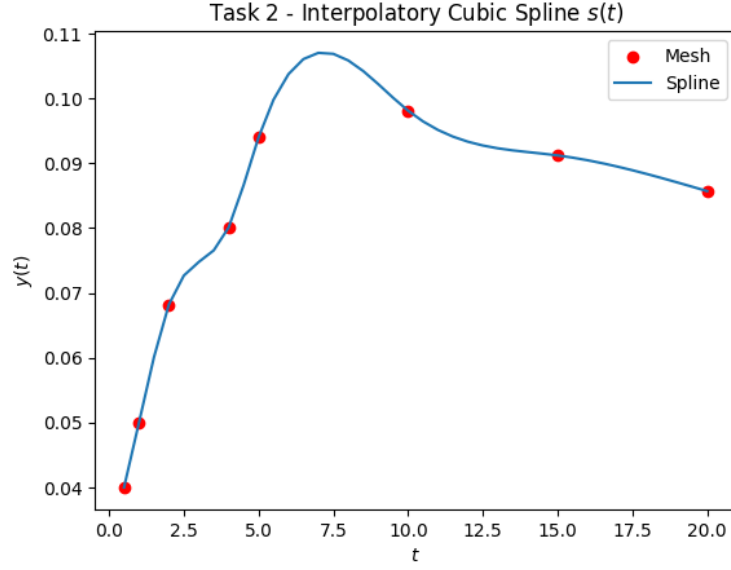


Figure 13. Given (t_i, y_i) pairs (in red) with a natural boundary condition interpolatory cubic spline.

Next, $f(t)$ was estimated, where $f(t) = y(t) + ty'(t)$. In order to obtain these estimates, the spline approximation for $y(t)$ was differentiated (using the formula given in Set 17 of Dr. Gallivan's class notes). Table 3 displays the computed estimates for $f(t)$ from $t_1 = 0.5$ to t_{40} with increment $\Delta t = 0.5$.

t	$f(t)$	t	$f(t)$	t	$f(t)$
0.5	0.04990	1.0	0.07038	1.5	0.08862
2.0	0.09355	2.5	0.08736	3.0	0.08437
3.5	0.09282	4.0	0.12098	4.5	0.15362
5.0	0.16214	5.5	0.15289	6.0	0.14080
6.5	0.12686	7.0	0.11207	7.5	0.09743
8.0	0.08393	8.5	0.07258	9.0	0.06436
9.5	0.06028	10.0	0.06134	10.5	0.06538
11.0	0.06947	11.5	0.07343	12.0	0.07708
12.5	0.08023	13.0	0.08270	13.5	0.08431
14.0	0.08487	14.5	0.08421	15.0	0.08214
15.5	0.07931	16.0	0.07651	16.5	0.07377
17.0	0.07111	17.5	0.06858	18.0	0.06620
18.5	0.06399	19.0	0.06199	19.5	0.06023
20.0	0.05874				

Table 3: Values of t and $f(t)$

Finally, $D(t)$ was estimated, where $D(t) = e^{-ty(t)}$. In order to obtain these estimates, each $y(t)$ value was multiplied by $-t$. Each $D(t)$ value is then computed as e raised to the power to the corresponding product. Table 4 displays the computed estimates for $D(t)$ from $t_1 = 0.5$ to t_{40} with increment $\Delta t = 0.5$.

t	$D(t)$	t	$D(t)$	t	$D(t)$
0.5	0.98020	1.0	0.95123	1.5	0.91384
2.0	0.87249	2.5	0.83388	3.0	0.79908
3.5	0.76495	4.0	0.72586	4.5	0.67719
5.0	0.62500	5.5	0.57759	6.0	0.53664
6.5	0.50188	7.0	0.47277	7.5	0.44866
8.0	0.42880	8.5	0.41239	9.0	0.39857
9.5	0.38642	10.0	0.37494	10.5	0.36325
11.0	0.35120	11.5	0.33887	12.0	0.32635
12.5	0.31376	13.0	0.30123	13.5	0.28890
14.0	0.27692	14.5	0.26544	15.0	0.25462
15.5	0.24454	16.0	0.23520	16.5	0.22653
17.0	0.21847	17.5	0.21098	18.0	0.20399
18.5	0.19746	19.0	0.19134	19.5	0.18558
20.0	0.18014				

Table 4: Values of t and $D(t)$

If we were to instead use a piecewise polynomial $g_d(x)$ to obtain estimates for these values, things would be a little different. A piecewise linear polynomial would simply connect the dots, drawing a straight line between consecutive given mesh points. A piecewise quadratic polynomial would use intervals of 3 points to generate local polynomials. However, this would leave us with one point left over. To address this issue, we could potentially use overlapping supports to generate the polynomials. A similar issue would be ran into when constructing a piecewise cubic polynomial. We would need overlapping supports for the local polynomials in order to make this work. As the degree used increases, the smoother and more accurate the approximation would be.

6 Conclusions

This report explored various interpolation techniques, including global and piecewise polynomial interpolation, cubic Hermite interpolation, and interpolatory cubic splines, to approximate both continuous functions and discrete datasets. The results demonstrated that interpolation accuracy depends on factors such as the choice of basis functions, mesh selection, and function behavior. While global polynomial interpolation provided reasonable accuracy, it exhibited oscillatory errors, particularly for highly oscillatory functions. Piecewise polynomial interpolation and cubic splines generally produced more stable and accurate approximations, with B-spline parameterization and Chebyshev meshes reducing interpolation errors in certain cases.

For the discrete dataset, the interpolatory cubic spline with natural boundary conditions effectively estimated function values and related quantities. The accuracy of these estimates further supports the use of splines for data fitting, particularly when smooth approximations are required.

Overall, this study highlights the strengths and limitations of different interpolation methods, demonstrating that careful selection of interpolation techniques is crucial for achieving accurate approximations in numerical analysis.

7 References

In the creation of this report, I referenced Dr. Gallivan's class notes. I also consulted with Brayton Link and Dan Mazus regarding algorithm development.